Operator and Representation Layers for Computational Materials Science: Principles and First-Demonstration Plan

G. Barbalinardo
University of California, Davis

May 29, 2026

Abstract

This document records the design of a typed operator/representation framework for computational materials science workflows, organized around a small set of principles. Workflows are directed acyclic graphs of *operator physics states* connected by *operator operations*, and concrete numerical results are *representations*—typed projections of operator states into a discretization scheme. The operator root is the *Born–Oppenheimer potential* of a material, expanded as a Taylor series in atomic displacements; force constants of every order are derived states obtained by extracting derivatives of this root and truncating the expansion at a chosen order. Every state carries a provenance chain that records the operations along its path; every representation carries a discretization scheme. Approximation and discretization errors live in disjoint type universes and compose along graph edges; the machinery for tracking error formulas symbolically and composing them across the graph is designed but its full implementation is deferred to Phase 2. As a first demonstration, three established lattice thermal-transport codes (kaldo, phono3py, ShengBTE) are reframed as distinct representations of a single operator DAG, with cross-code reconciliation at canonical intermediate states as the Phase 1 deliverable. Phase 1 stubs the upstream operations that produce force constants from the potential—declaring the relevant operator types and provenance labels but not modeling them in detail—so that Phase 2 can elaborate the upstream without restructuring the framework. The document is intended as the working architectural reference for the `openmaterials-ai` project; it covers each architectural decision, the alternatives considered, and what we deferred.

Contents

1 Principles

The operator layer is built on a small set of commitments. They are stated here without their alternatives or justifications; supporting definitions follow in later sections.

1. **Two worlds, one bridge.** The operator layer keeps the operator physics world (typed states, operator operations, approximation errors) strictly separate from the numeric world (discretized arrays, computed values, discretization errors). They are connected by exactly one bridge: the representation functor.

2. **Operator states are unit-free.** Dimensionful quantities (frequency, energy, length, time) exist at the operator layer, but specific unit choices (THz vs eV vs rad/s, Å vs Bohr) do not. Unit choice belongs to the representation and is declared by the adapter that produced it; the framework normalizes to a canonical unit before any cross-representation comparison.

3. **The atom is a operator operation.** The atomic unit of meaning is a typed transformation between operator states, not an instruction in an input file or an algorithm. Every operation carries a *operator formula*—a sympy expression (closed-form output) or sympy.Eq (implicit equation defining the output)—stating what it computes. The formula is the operator claim, machine-readable and Lean-projectable, against which adapter conformance can be checked.

4. **States are typed witnesses; nodes are ObservableSpaces or HiddenSpaces.** A state is the typed claim that a operator physics object exists with a particular history. The operator layer splits states into two structural kinds: *ObservableSpaces* (gauge-invariant, first-class, required to agree across adapters) and *HiddenSpaces* (gauge-dependent intermediate scaffolding whose per-element values reflect basis / phase / BZ-summation choices

the framework does not pin down). Each state declares fields, and each field has an index signature (e.g. $\omega(\mathbf{q}, \nu)$ has indices $(\mathbf{q}, \nu)$; $\kappa^{\alpha\beta}$ has (α, β)). Numerical content lives in the representation, separate from the witness.

5. **Workflows are directed acyclic graphs.** A workflow is a graph of states connected by operations, not a linear sequence. Cross-code comparison reduces to graph alignment over the same operator template.

6. **Identity is type plus provenance.** A operator state is identified by its physics type and the ordered chain of operations that produced it. Two states with the same type but different histories are different states.

7. **Operation identity is parameterized.** An operation is identified by its name together with the parameters that affect physics. Different physics-affecting parameter values produce different operations and therefore different output states. Function-family parameterizations (e.g., Gaussian broadening by standard deviation vs. by half-width) are canonicalized at the operator layer to a single choice; adapters translate to their internal convention.

8. **Discretization belongs to representation.** Mesh densities, supercell sizes, displacement steps, and broadenings live on the representation, not on the operator state. Convergence is a relationship between a operator state and a family of its representations.

9. **Cross-code agreement is required at ObservableSpace nodes only.** Two adapters' representations of an ObservableSpace must agree (after unit and normalization conversion) to within the observable's declared tolerance. Representations of a HiddenSpace are not cross-code comparable per-element; the operator layer's `compare` returns the status `NOT_COMPARABLE` and reports residuals as diagnostic information only. To make a HiddenSpace cross-code comparable, contract it into an ObservableSpace (e.g., per-mode $\Gamma_{q\nu}$ is a HiddenSpace; $\sum_{q\nu} \Gamma$ contracted via the BZ sum is an ObservableSpace that does agree).

10. **Errors are designed-for.** Approximation errors live on operations; discretization errors live on representations. The type system reserves slots for both; operator composition of error formulas is a Phase 2 capability.

11. **The operator root is the potential.** Every representation in the framework ultimately traces back to the Born–Oppenheimer potential of the material. Force constants of every order are derived states obtained by extracting derivatives of the potential and truncating the Taylor expansion. The upstream is stubbed in Phase 1.

12. **Sources are produced by nullary operations.** Source states (the potential, but also temperature, lattice constant, etc.) are the outputs of nullary `provide_*` operations rather than nodes with empty provenance. This unifies the DAG: every node is the output of some operation, and the operator layer has no special "input" category. The representation of a source state carries the externally supplied value (e.g., $T = 300\,\mathrm{K}$); the operation is just a structural tag.

13. **Lean-compatible by structure.** The operator layer is implemented in Python but designed so that its types transliterate cleanly into Lean. Three disciplines protect this option: closed unions for physics types, no Python-encoded physics invariants, operation parameters always recorded in output state metadata.

2 Background and Motivation

Modern computational materials science has many codes, many workflows, and many opinions about how to compose them. Despite a generation of effort on workflow managers (AiiDA, Atomate, FireWorks, Atomate2, Aflow), one quality is still missing: a workflow should expose, as typed and composable data, *the physics it computes*.

Today the standard unit of shared meaning between codes is the *input file*: an ordered sequence of instructions that, together with a particular code's internal logic, produces an output. This unit is opaque to two questions:

- **Given a result, what physical assumptions and approximations does it embed?** An input file declares parameters but not their meaning. A k-mesh is just a triple of integers; the relationship between that mesh and the convergence of the integrated observable is not part of the file.
- **Given two results from two codes, where exactly do they diverge?** Identical force constants run through kaldo and through ShengBTE typically yield slightly different thermal conductivities. The difference is not localized to a particular operation, even though—in principle—it is the result of distinct computational choices at specific steps.

A previous attempt of ours, `MaterialsCodeGraph` (MCG), addressed parts of this gap by treating each instruction in an input file as the atomic unit of a knowledge graph and tracking inputs, parameters, and outputs at that level. MCG's tool-encapsulation principle (all tool knowledge in YAML/Markdown configs, all code generic over tool) was the right design at that level of abstraction. But the level of abstraction itself is too low: an instruction in an input file is a syntactic unit, not a semantic one. It tells you what a code is told to do, not what *physics* the code thereby computes. Two codes that compute the phonon dispersion via diagonalization of the dynamical matrix express that operation as different instructions; yet they perform the same physics. A graph indexed by instructions sees them as two unrelated tasks.

The operator layer proposed here *starts from the physics* and treats input files as derived artifacts. The atomic unit is no longer an instruction; it is a *operator operation* between typed physics states. The states are the load-bearing entities: a state is what changes when an operation is applied, and a workflow is the graph that records those state transitions. Concrete numerical computation is then a separate, typed projection of the operator workflow into a discretization scheme.

This is a deliberate departure from the input-file paradigm. We argue below that it pays for the extra abstraction with three concrete returns: cross-code reconciliation at canonical intermediate states, derived (rather than empirical) error bounds on computed observables, and a clean foundation for future formal verification.

3 Architectural Principle: Two Worlds, One Bridge

The single most important design decision is a strict separation between two type universes:

- **The operator world** holds typed physics states (Hamiltonians, dispersion relations, scattering rates, distribution functions, partition functions) and operator operations between them. Errors here are *approximation errors*—the $O(\cdot)$ terms introduced by truncating perturbation series, linearizing equations, or imposing physical assumptions such as the Born–Oppenheimer or harmonic approximation. The operator world contains no numerical content. Two operator states that differ only in their numerical estimates are the same operator state; two operator states that differ in the chain of approximations leading to them are different operator states.

- **The numeric world** holds discretized realizations of operator states: arrays sampled on finite grids, mode-resolved quantities at finite k-meshes, distributions evaluated at finite temperature. Errors here are *discretization errors*—the $O((1/N)^k)$ or $O(h^k)$ terms introduced by sampling a continuum quantity on a finite scheme. The numeric world contains no operator content; it does not know what theory the numbers are estimates of.

The two worlds are connected by exactly one bridge, the *representation functor*, which takes (i) a operator state and (ii) a discretization scheme and produces a numeric representation. Approximation errors compose in the operator world along provenance chains; discretization errors compose in the numeric world along execution graphs; the two combine only at representation time, where the total error of a numerical result decomposes into its operator and numeric contributions.

This separation is load-bearing. It is also somewhat unusual. Most computational physics codes mix the two worlds: a "Hamiltonian" object in such a code is sometimes a typed mathematical operator and sometimes a 4-D `numpy` array. The operator layer refuses to do this. If you are holding a Hamiltonian, you know which world you are in and you cannot accidentally mix the two. This refusal is what makes errors composable, theorems statable, and cross-code comparisons well-defined.

3.1 The full chain: two layers plus the external codes

In implementation terms, the project has *exactly two layers* under its own control plus the external physics codes that produce numerical data. Nothing else is a layer: tests, visualization, and experiment scripts are supporting infrastructure.

Layer 1 — operator. Types, declarations, formulas, gauges. Machinery in `omai.operator` (the `Space` and `Operator` types, `Field`, `Dimension`, `GaugeAction`, `SymmetryGroup`, `validate_dag`); domain instances in `omai.thermal_transport.operator` (the 46-node, 47-edge DAG with sympy formulas on each edge, gauge actions). No values, no units, no per-code knowledge.

Layer 2 — representation. Per-code declarations plus the runtime that operates on actual code outputs. Machinery in `omai.representation` (the `SpaceRepresentationSpec` and `OperatorRepresentationSpec` types, `Unit` and unit conversions, the `Representation` runtime data class, `compare()` with its five-status verdicts); domain instances in `omai.thermal_transport.represente` (one file per code: `kaldo.py`, `phono3py.py`).

External codes. kaldo, phono3py, ShengBTE: do the actual physics computation in their own native code, output arrays in their native units and normalizations.

The chain in motion. A typical cross-code verification flows through these layers like this:

1. User invokes the codes externally (e.g. kaldo and phono3py), which produce raw output arrays in their own formats.
2. User wraps each array via `represent(spec, field_name, array)`: this validates the field exists on the operator state and produces a `Representation` object tagged with the right adapter spec.
3. User calls `compare(m_a, m_b)`. `compare` consults the operator layer for the cross-code factor (units × normalizations), applies it, optionally contracts, and consults the operator layer again for the space's gauge kind to decide whether an AGREE/DISAGREE verdict is even meaningful or the comparison is NOT_COMPARABLE.

4. User gets a `RepresentationComparisonResult` with status, residuals, and the applied factor.

No additional layers, no other indirection. The two two layers expose their content in matched pairs (machinery in `omai/<layer>/`, domain instances in `omai/<domain>/<layer>/`), and the codes attach at the edge.

4 Formal Definitions

We collect the formal definitions implied by the decisions above.

4.1 Operator state

A operator state s is unit-free: its physics type carries the dimension of any quantity it represents (a frequency, an energy density, a tensor of given rank), but no unit choice. Numerical content and units appear only at representation time.

A operator state s is a tuple (κ, τ, π, F) where:

- $\kappa \in \{\textsc{ObservableSpace}, \textsc{HiddenSpace}\}$ is the *gauge-invariance kind*: ObservableSpaces are gauge-invariant and cross-code comparable; HiddenSpaces carry gauge freedom (eigenvector phase, BZ-summation choice, degenerate-subspace rotation, ...) that the operator layer doesn't pin down, so their per-element values differ across adapters by design;
- τ is a *physics type* drawn from a finite registry. Derived types in the thermal-transport scope include `ForceConstants[order=`n`]`, `DynamicalMatrix`, `Frequency`, `Eigenvectors`, `GroupVelocity`, `HeatCapacity`, `Linewidth`, `MeanFreeDisplacement`, `ThermalConductivity`; sources include `Potential`, `Temperature`. Some types are further parameterized by upstream operation choices (e.g., `ThermalConductivity[bte_solver= rta]` is a HiddenSpace since RTA's $1/\Gamma$ non-linearity breaks gauge invariance, whereas `ThermalConductivity[bte_solver=direc` is an ObservableSpace because the full LBTE preserves it);
- π is the *provenance*: an ordered list $[\mathcal{O}_1, \mathcal{O}_2, \ldots, \mathcal{O}_n]$ of operator operations whose composition produced s. Provenance is never empty—source states have a single nullary operation (`provide_potential`, `provide_temperature`, ...);
- $F = (f_1, f_2, \ldots)$ is the state's tuple of *fields*. Each field has a name, a dimension (frequency, energy, length, ...), and an index signature (e.g., ω has indices $(\mathbf{q}, \nu)$; v has $(\alpha, \mathbf{q}, \nu)$; κ has (α, β)).

Two operator states are equal iff their names match. The composed approximation error on s (a Phase 2 capability) would be

$$\epsilon_{\text{approx}}(s) = \text{compose}\big(\epsilon_{\mathcal{O}_1}, \epsilon_{\mathcal{O}_2}, \ldots, \epsilon_{\mathcal{O}_n}\big). \tag{1}$$

Source versus derived states. Operator states fall into two kinds, both produced by some operation:

- *Source states* are the outputs of *nullary* operations (`provide_potential`, `provide_temperature`, ...): operations with no operator-state inputs. They are empirical inputs (lattice constant, temperature, pressure, applied field), the operator `Potential` itself (Phase 1 stubs the upstream of the potential), or measured observables (a thermal conductivity reported from a TDTR experiment, a Raman peak). The representation of a source state carries the externally supplied value (e.g., 5.43 Å for the lattice constant of silicon, or $T = 300\,\text{K}$); its discretization scheme is trivial (a singleton choice rather than a refinable mesh) but its error is still meaningful (experimental uncertainty, or definitional precision).

- *Derived states* are the outputs of operations that consume one or more upstream states. Examples: `ForceConstants[order=`n`]` (derived from `Potential` via `compute_force_constants[order=`n`]`, currently stubbed), `Frequency` and `Eigenvectors` (multi-output of `compute_dispersion`), `Linewidth`, `MeanFreeDisplacement`, the computed κ.

Both kinds follow the same machinery for representation. The unification of source and derived states under the same "output of an operation" rule answers several otherwise awkward cases: empirical inputs ("where does the lattice constant live?"), experimental observables ("how does a measured $\kappa(T)$ enter the framework?"), and the unmodeled operator root in Phase 1 ("what is the silicon potential, before we differentiate it?"). All three are representations of source states whose provenance is a single nullary operation; the second is what makes the unified theory–experiment comparison framework native rather than bolted on; the third is how Phase 1 stubs the upstream of `Potential` without yet modeling it in detail.

What "witness-as-state" means, concretely. The phrase *witness-as-state* comes from the Curry–Howard correspondence in type theory: a typed value is read as evidence (a "witness") for a proposition. We borrow the intuition without borrowing the formal proof obligations. In this framework a state is not the data it contains; it is the *claim* that something exists, expressed in a typed form rich enough to carry meaning.

Concretely, three layers are conflated in most computational physics codes; the operator layer keeps them apart:

1. **The proposition** – "the phonon dispersion of crystalline silicon under the harmonic approximation, derived by truncating the Born–Oppenheimer Taylor expansion at second order, exists as a function $\omega(\mathbf{q}, \nu)$ from the Brillouin zone to positive reals."

2. **The witness** – a typed object `s = (Dispersion,` π`)` that records precisely this claim. The physics type `Dispersion` encodes the kind of object being asserted to exist; the provenance π records the chain of operations (extraction, truncation, Fourier transform, eigendecomposition) that produced it. The witness carries no numerical content; it is the assertion that such an object exists with this history.

3. **The representation** – a tuple $(s, \Sigma, x, \epsilon_{\text{disc}})$ in which x is a concrete 4-D `numpy` array of frequencies sampled on a chosen k-mesh, Σ is the discretization scheme, and ϵ_{disc} is the associated discretization error. The representation is the numerical evidence that the witness asserts to exist.

A code that conflates these layers (treats the dispersion *as* the array) cannot distinguish "the same physics computed two different ways" from "different physics computed the same way." A framework that keeps them apart can: two witnesses with identical types but different provenance are different states, regardless of whether their representations happen to have similar numerical content; one witness with two representations on different k-meshes is the same state, regardless of how different the arrays look numerically.

This is the load-bearing distinction. It is also the answer to several otherwise hard questions. "What is the dispersion of silicon?" is the question the witness answers. "What does the dispersion of silicon look like on a $14 \times 14 \times 14$ mesh in kaldo" is the question its representation answers. "Are these two computed dispersions the same?" resolves into two sub-questions—are the witnesses equal (a question about provenance), and are the representations consistent within their discretization errors (a question about numerics)—each answerable without confusion with the other.

Read in this light: the operator states described elsewhere in this document are all witnesses: they assert the existence of a particular physics object derived in a particular way. The numerical content, when present, lives in the representation, separated from the witness by the operator/represented boundary. Operations between operator states are operations between witnesses, with no numerics involved; they are essentially structural moves on a graph of typed assertions.

4.2 Operator operation

A operator operation $\mathcal{O}$ is a typed map between operator states,

$$\mathcal{O} : \tau_{\text{in}_1} \times \tau_{\text{in}_2} \times \cdots \to (\tau_{\text{out}_1}, \ldots, \tau_{\text{out}_m}), \tag{2}$$

equipped with three pieces of metadata:

- a *operator formula* $\mathcal{F}_\mathcal{O}$ stating what the operation computes, encoded as a sympy expression for closed-form ops or a sympy.Eq for implicit ones (eigenvalue equations, linear systems). *Every* operation carries a formula—source operations have identity formulas (e.g., `provide_temperature`: $T = T_{\text{provided}}$);
- an *approximation error formula* $\epsilon_\mathcal{O}$, a operator expression in physical small parameters (e.g., λ for the strength of a perturbation, T/T^* for a Sommerfeld expansion). Exact operations have $\epsilon_\mathcal{O} = 0$; approximating operations have $\epsilon_\mathcal{O} = O(p^k)$ for some parameter p and order k;
- a possibly empty set of *schemes* (per §4.2) and *parameters* (dimensioned but unit-free at the operator layer).

Multi-output operators (e.g. `compute_dispersion` producing `Frequency` and `Eigenvectors` together) are first-class. The formula $\mathcal{F}_\mathcal{O}$ is the operator layer's machine-readable claim about what the operator produces: adapter conformance can be expressed as "the adapter's runtime behavior matches $\mathcal{F}_\mathcal{O}$ under its declared unit, normalization, and scheme choices," in principle automatable rather than reverse-engineered from kernels.

Operation identity is parameterized. Several operations in the operator layer carry method choices that affect physics: the scattering processes included (3-phonon only, or 3-phonon plus isotopic, or with boundary scattering), the Boltzmann solver (relaxation-time approximation, iterative, direct inversion), the broadening scheme (Gaussian, Lorentzian, adaptive), and so on. The operator layer treats these as part of the operation's *identity*, not as runtime configuration that gets discarded.

Concretely, an operation is identified by the tuple (operation name, parameters affecting physics). Two invocations of `compute_scattering_rates` with different `scattering_processes` are different operations; they appear differently in the provenance, and they produce operator states that are formally distinct (different type metadata, different total approximation error). The *name* `compute_scattering_rates` is the same; the *parameterized identity* differs.

This convention has three consequences. First, the vocabulary of operations remains small (one entry per named transformation, not one entry per parameter combination), but the provenance remains expressive (because provenance records the full parameterized identity). Second, output states' types are mildly dependent: `ScatteringRates` is parameterized by the scattering-processes choice, even though the operator layer does not require Python's type system to enforce this dependence. Third, the operator layer is structurally compatible with dependent type theory: when transliterated to a system like Lean, the parameter values lift to type-level indices and the dependence becomes formal.

4.3 Operator workflow

A operator workflow is a finite directed acyclic graph $W = (V, E)$ where V is a set of operator states and E is a set of operator operations. The graph is rooted at one or more *source* states (typically empirical inputs such as the crystal structure) and terminates at one or more *observable* states (e.g., the lattice thermal conductivity).

4.4 DAG extension rules

When a new physical quantity or new way of computing an existing quantity must be added to the DAG, three patterns are available. The choice has structural consequences, so we record the rules here rather than leaving them to taste.

Where formulas live. *Edges* carry the sympy formula and the declared schemes; *spaces* are typed places (a claim that "this physical quantity exists, with this dimension, this gauge type, this label vocabulary"). Different production formulas do not, by themselves, force different spaces — they force different edges.

Pattern A — label on the space. Use when the label changes the *gauge type* (ObservableSpace vs. HiddenSpace) of the produced quantity, *and* the labelled space is terminal (or its labelled consumers are themselves a closed sub-branch). Existing examples:

- `MeanFreeDisplacement[bte_solver=rta|direct_inverse]` — RTA is HiddenSpace, direct is ObservableSpace. Propagates to `ThermalConductivity[bte_solver=...]` and stops there.
- `ThermalConductivity[transport_model=lbte|wigner|qhgk]` — adds the orthogonal axis for Wigner and QHGK transport. All terminal.

This pattern is safe *only* for terminal or near-terminal nodes. Putting a type parameter on an intermediate state forces every downstream consumer to be parameterised in turn, polluting the entire downstream sub-DAG.

Pattern B — sibling states converging through an explicit edge. Use when several variants represent physically distinct contributions, with different inputs, that must combine before a downstream consumer uses them. The variants are *sibling named states*; a converging edge produces the combined "total" state, and the downstream sees only the total. Existing example:

- `AnharmonicLinewidth` (H, inputs FC^3, e, ω, T), `IsotopicLinewidth` (H, inputs IsotopeAbundances, e, ω), `BoundaryLinewidth` (H, inputs v, length scale) all flow into `TotalLinewidth` via the `sum_linewidths` edge. `solve_bte_*` consumes only `TotalLinewidth`.

The advantage over Pattern A here is that the per-channel inputs differ; sibling states make that visible in the DAG diagram and let adapters declare the channels they support independently.

Pattern C — shared output node, alternative producing edges. Use when several formulas produce the *same-typed* output with the same gauge classification, but from different inputs. The downstream is unaware of which path was taken; provenance and adapter specs record it. A literal in-place modifier — a single state with an edge that consumes and produces itself — would create a cycle and is rejected by the DAG validator. The canonical workaround is to introduce a small intermediate node that names the pre-modification version. Existing example:

- `compute_dynamical_matrix(FC²)` → `BareDynamicalMatrix`, followed by either `apply_nac_correction(` `BornCharges,` ε_∞`)` → `DynamicalMatrix` (polar branch) or the identity edge `identity_dm(BareDM)` → `DynamicalMatrix` (non-polar branch). `compute_dispersion` consumes only `DynamicalMatrix` and does not need to know which producing edge fired.

Decision flow. When adding a variant of an existing quantity:

1. Does the variant change the gauge type? *Yes:* Pattern A.
2. Does the variant have different inputs that physically combine? *Yes:* Pattern B.
3. Same gauge type, same output type, alternative production? *Yes:* Pattern C.

In all three patterns, *edges carry the formulas* and *states carry the typed places*; the patterns differ only in which nodes are shared and which are distinct.

Two-tier validation. Validation runs at two layers. *Declarations* are checked at module load: `validate_dag` on `NODES` and `EDGES` verifies the gauge-discipline invariants (every HiddenSpace declares its gauge group and gauge-invariant contractions; scheme names referenced by adapters exist on their operators; no cycles), and now also the sympy-layer invariants (every edge's `formula.free_symbols` are derivable from its inputs and parameters; LHS indices match the output space's field; auxiliary formulas close over the main formula's vocabulary). *Cross-code data* is checked at comparison time: `compare_operators` on a pair of `SpaceRepresentationSpec`s verifies that two codes are claiming the same operator (compatible units, canonicalisable normalizations), and `compare_representations` on a pair of `Representation`s applies the spec-derived conversion and emits the five-status `RepresentationComparisonRes` Mismatches at the declaration layer are caught before any code runs; mismatches at the data layer are caught at the operator↔representation boundary, never silently.

4.5 Discretization scheme

A discretization scheme Σ is a finite tuple of parameters $(N_1, \ldots; h_1, \ldots)$ specifying how a continuum quantity is sampled. Examples include the k-point mesh density, the supercell size, the finite-difference displacement step, the broadening width, and the integration order. The scheme also carries an error model $\epsilon_\Sigma(N_1, \ldots; h_1, \ldots)$ giving the asymptotic dependence of the discretization error on these parameters.

4.6 Representation

A representation m is a runtime data class wrapping one adapter's emission of one field on one space:

$$m = (\text{SpaceRepresentationSpec}, \texttt{field\_name}, x)$$

where SpaceRepresentationSpec carries the operator `Space` s, the representation name, and the adapter's declarations (the *unit* of each field and the *normalization* of each field, plus notes); `field_name` selects which field of s is represented; and x is a concrete `numpy.ndarray`. The discretization scheme Σ and discretization error ϵ_{disc} are deferred to Phase 2.

Why the representation spec ships with the data: unit × normalization. Different codes express the same physics under two independent multiplicative choices, and the spec records both:

- **Unit** (measure choice). kaldo's linewidth is in angular-frequency THz while phono3py's is in linear-frequency THz; kaldo's heat capacity is in J/K, phono3py's in eV/K. Each `Unit` entry carries a `to_operator` multiplier into the operator-canonical unit for its dimension.

- **Normalization** (definitional choice). kaldo emits $\Gamma = 2\,\mathrm{Im}\,\Sigma$ while phono3py emits $\Gamma = \mathrm{Im}\,\Sigma$; ShengBTE reads FC3 in the mixed-dimension form $\mathrm{eV}/(\mathrm{Å}^2 \cdot \mathrm{nm})$ while kaldo / phono3py read $\mathrm{eV}/\mathrm{Å}^3$. Each `Normalization` entry carries a `to_operator` multiplier into the canonical definitional form.

The two choices are orthogonal: a raw array without its spec is unsafe to compare against another; with the spec attached, the operator layer computes both factors mechanically and surfaces unit / normalization mismatches as type-level declarations rather than as numerical mysteries.

Star topology: the operator layer is the unique hub. The cross-representation conversion factor is *never* a direct representation-to-representation primitive. The two primitives are `representation_to_operator`(A, obs) and its inverse `operator_to_representation`(A, obs), each defined as a clean composition of the unit's and normalization's `to_operator` multipliers:

$$\texttt{representation\_to\_operator}(A, \mathrm{obs}) \;=\; \texttt{UNITS}[u_A].\texttt{to\_operator} \cdot \texttt{NORMALIZATIONS}[n_A].\texttt{to\_operator}.$$

The cross-adapter A→B factor is the composition through the operator hub,

$$\texttt{operator\_to\_representation}(B, \mathrm{obs}) \;\cdot\; \texttt{representation\_to\_operator}(A, \mathrm{obs}),$$

derived and never primitive. Geometrically the operator layer is the unique hub of a star topology with each representation as a spoke; an N-representation domain requires N spec sets to the operator hub, never N^2 pairwise conversions. Lean-side this is the right factoring: the operator layer is the *theory*, each representation is a *model*, and any representation-to-representation morphism factors through the theory by construction.

The same discipline applies edge-by-edge in the executor: when the operator-side runtime walks the DAG and dispatches an edge to code A (i.e. to representation A), the value returned MUST be carried back through A's `representation_to_operator` before the next edge runs — whether that next edge is sympy-executable (closed-form on the operator layer) or dispatched to a different representation B (which then applies B's `operator_to_representation`). Operator-form intermediates are the lingua franca between heterogeneous edges; no edge-to-edge handshake bypasses the operator hub.

4.7 Cross-representation comparison: `compare`

The bridge from spec to verified empirical fact is the function

$$\texttt{compare}(m_a, m_b; \mathrm{rtol}, \mathrm{atol}, \mathrm{contraction}) \;\to\; \textsc{RepresentationComparisonResult}$$

which takes two representations of the same space and field, applies the cross-representation conversion factor derived from their declared units and normalizations, optionally contracts both arrays via a user-supplied callable (e.g. `numpy.sum`), and reports residuals together with a *status* drawn from five values:

Expected_Agree the space is an ObservableSpace; the arrays agree within $(\mathrm{rtol}, \mathrm{atol})$. The operator layer's prediction holds.

Expected_Disagree the user predicted disagreement (override) and the arrays disagree. Useful for intermediate contractions of a HiddenSpace that the user knows is only partially gauge-invariant.

Not_Comparable the space is a HiddenSpace and no contraction was supplied. The operator layer refuses to make an agree/disagree verdict; residuals are still computed and returned for diagnostic inspection, but they carry no normative weight.

Unexpected_Disagree the space is an ObservableSpace but the arrays disagree. *Real anomaly*: either the spec is missing a normalization, or the codes genuinely compute different physics, or rtol is too strict.

Unexpected_Agree the user predicted disagreement and the arrays agreed. Rare; the per-element protocol may be tighter than declared.

The status taxonomy separates three things that earlier versions of this framework conflated: the operator layer's *prediction* (would these agree?), the empirical *outcome* (did they?), and whether the comparison is even *meaningful* for the given state kind. Only UNEXPECTED_DISAGREE flags an anomaly that needs investigation; EXPECTED_AGREE and EXPECTED_DISAGREE are successful predictions; NOT_COMPARABLE is a HiddenSpace saying "don't ask this question."

4.8 The validation engine: execute, compose, cross-check

The representation layer carries a runtime that *runs* the operator DAG, not merely compares emitted arrays. `compute(target, sources)` lazily resolves a target space: a space with a registered source is loaded and lifted to operator form (units and normalizations applied automatically), and every other space is derived by executing its producing operator's sympy formula via `apply_edge`. Closed-form paths can also be fused symbolically (`compose_executable`) into one synthetic edge; the composed-then-executed value must equal the edge-by-edge value, so the symbolic composer and the numerical executor validate each other. Finally `cross_check` computes a target by several redundant routes (different codes per leaf, or different operator paths) and pairwise-compares them, with agree/disagree verdicts governed by the target's Observable/HiddenSpace typing: routes to an Observable must agree, routes through a HiddenSpace need not. On Si-Tersoff the engine derives molar heat capacity from frequencies and matches phonopy's emitted value to 0.1%, and contracts κ_{LBTE} from loaded group velocity, mean free displacement, and a derived heat capacity, matching kaldo's emitted conductivity.

The executor reconciles units dimensionally: each dimension's canonical unit carries an absolute SI scale, and when an operator's formula is a pure contraction (a monomial in its input fields and declared parameters), the executor rescales the raw canonical-unit result into the output's declared canonical unit automatically. This is what lets the κ_{LBTE} contraction of Å/THz-canonical group velocity and mean free displacement with an $\mathrm{Å}^3$ cell volume emerge directly in $\mathrm{W}/(\mathrm{m}\cdot\mathrm{K})$, with no hand-applied conversion. Closed-form edges (whose constants already carry the SI conversion) and additive edges are not monomials and are left untouched.

4.9 HiddenSpace discipline and gauge actions

The ObservableSpace / HiddenSpace distinction is load-bearing, so the operator layer enforces a discipline on how HiddenSpaces are declared. Each HiddenSpace carries three additional fields:

- `gauge_group`: a named identifier for the gauge equivalence acting on this state (e.g. "U(1)_phase_on_eigenve "bz_summation_permutation").
- `kind`: one of `scaffolding` (the HiddenSpace is consumed by a downstream operation producing an ObservableSpace, so the gauge orbit is eventually summed away) or `approximation` (the HiddenSpace is terminal — an approximation of an ObservableSpace that happens to break gauge invariance, with no downstream recovery; κ`[bte_solver=rta]` is the canonical example).
- `gauge_invariant_contractions`: names of ObservableSpaces in the DAG that capture this state's gauge-invariant content (empty for `approximation` kind).

A `validate_dag` function walks the node and edge sets at load time and rejects: undeclared gauge groups, invalid kinds, scaffolding HiddenSpaces with empty or non-resolving contractions,

approximation HiddenSpaces that incorrectly declare contractions, and edges that reference unknown nodes. The thermal-transport DAG passes this validator; tests check it on every CI run.

Operator invariance via `GaugeAction`

Beyond named declarations, the operator layer supports *operator proofs* of gauge invariance for the tractable cases. A `GaugeAction` is a sympy-encoded transformation: a pattern to match (e.g., $e_{i,q,\nu}$) plus a transform (e.g., $e^{i\theta_{q,\nu}} \cdot e_{i,q,\nu}$). Given an operation's sympy formula, the operator layer substitutes the gauge action and runs `sympy.simplify` on the difference; if the result is zero, the formula is machine-verified invariant.

Example: applying the U(1) phase $e_{i,q,\nu} \to e^{i\theta_{q,\nu}} e_{i,q,\nu}$ to

$$v^{\alpha}_{q,\nu} = \frac{1}{2\omega_{q,\nu}} \sum_{i,j} e^{\dagger}_{i,q,\nu} \frac{\partial D_{ij}}{\partial q^{\alpha}} e_{j,q,\nu}$$

gives $e^{-i\theta_{q,\nu}} \cdot e^{i\theta_{q,\nu}} = 1$, and sympy mechanically reduces the difference to zero. This is the operator layer's first machine-verified physics claim — per-mode group velocity is U(1)-phase invariant, by construction of the formula.

What's tractable today: simple operator substitutions (U(1) phase), finite group actions encoded as patterns over indexed expressions (crystal point groups), and other gauges where the transformation is algebraically substitutable. *Not tractable today:* continuous Lie group actions on subspaces (e.g., U(d) on degenerate-subspace), and data-dependent gauges (e.g. degenerate-subspace rotation that only acts at degenerate ω). For those, the operator layer falls back to the Level 1 named-gauge declarations.

Crystal symmetry as a operator declaration. Crystal symmetry at the operator layer level is purely a *declaration*: a `SymmetryGroup` carries a Hermann-Mauguin / Schoenflies name (`Oh`, `D6h`, `Ci`, ...) and an order $|G|$. No element data — no rotation matrices, no translation vectors, no group-element enumeration — lives in the operator layer.

The concrete handling — extracting symmetry from a structure, applying group operations to FC tensors, building irreducible-BZ reductions — belongs to the materials codes (phonopy, kaldo, ShengBTE, ...), typically via spglib. The operator layer's role is just to *name* the group so that operations can declare which symmetry they assume as a parameterized identity (`compute_force_constants[order=2, symmetry=Oh]` differs from `[..., symmetry=C1]`), so adapter specs can declare which group a particular run assumed, and so cross-code comparison can refuse to compare two representations whose declared symmetry groups disagree.

This is the same pattern `Potential` follows: declared symbolically as a labelled source state without the operator layer modeling its functional form. "How $\Phi^{(2)}$ is computed under crystal symmetry" is a code concern; "which symmetry group was assumed" is a operator-level declaration.

4.10 Representation functor

A representation functor $\mathcal{F}$ is associated with a particular code (kaldo, phono3py, ShengBTE). Given a operator workflow W and a discretization scheme Σ,

$$\mathcal{F}(W, \Sigma) \tag{3}$$

produces a directed graph of representations whose vertices are in one-to-one correspondence with a subset $V_{\mathcal{F}} \subseteq V(W)$ and whose edges are the computational realizations of the operator operations restricted to $V_{\mathcal{F}}$. Different codes implement different functors over the same operator workflow; their represented graphs are aligned by their shared operator template.

4.11 Total error

The total error on the representation m of a operator state s is

$$\epsilon_{\text{total}}(m) = \text{compose}\big(\epsilon_{\text{approx}}(s),\, \epsilon_{\text{disc}}(m)\big). \tag{4}$$

The composition rule depends on the operation type but is in all cases derivable from the typed metadata. The two contributions $\epsilon_{\text{approx}}(s)$ and $\epsilon_{\text{disc}}(m)$ are recovered separately, so a result that is converged in discretization but uncertain in approximation is distinguishable from one that is fully approximated but undersampled.

5 Lean-compatibility disciplines

"Lean-compatible" is structural rather than syntactic. The operator layer's Python implementation will not be Lean code, but it should be *shaped* so that the Lean version is a transliteration rather than a redesign. We commit to three disciplines that protect this option.

1. **Closed unions for physics types.** The registry of physics types (`Potential`, `ForceConstants`, `Dispersion`, `ScatteringRates`, ...) is exhaustive and closed—implemented in Python via sealed enums or tagged unions, not via open class inheritance. Lean represents these as inductive types with a fixed list of constructors; closed unions in Python preserve this structure exactly. Open inheritance would require redesign at the Lean stage.

2. **No physics invariants encoded in Python types.** Properties such as "a force-constant tensor is symmetric under the appropriate index exchanges" or "the eigenvectors of a dynamical matrix are orthonormal" are documented as informal invariants on each type but *not* enforced by the Python type system. In Lean these invariants become formal proof obligations attached to the corresponding types. Trying to encode them in Python (via runtime assertions or pydantic validators) creates a layer of checks that does not transliterate and risks coupling the implementation to Python-specific idioms.

3. **Operation parameters always recorded in output state metadata.** When an operation carries parameters that affect physics (§4.2), Python's type system will not enforce the dependence of the output type on those parameters, but the output state's metadata must still record them. In Lean these parameters lift to type-level indices, and the type-level dependence is enforced formally; in Python they are runtime metadata that downstream operations and provenance machinery treat as part of the operation's identity.

These disciplines are inexpensive to follow from day one and prohibitive to retrofit. They keep the transliteration cost low without committing the operator layer to Lean tooling we do not yet need.

6 First Demonstration: Lattice Thermal Transport

We instantiate the operator layer on the canonical workflow for lattice thermal conductivity κ, computed from second- and third-order interatomic force constants via the linearized Boltzmann transport equation.

6.1 The operator DAG

The operator DAG has 46 nodes and 47 edges. Two source nodes (`Potential`, `Temperature`) are produced by nullary `provide_*` operations; the `MeanFreeDisplacement` and `ThermalConductivity` states are each split into two parameterized variants by the upstream `bte_solver` choice (one

gauge-invariant, one not). A second tier of derived observables — `PhononDOS`, `Gruneisen`, `PhaseSpace3Phonon`, `VolumetricHeatCapacity`, `MolarHeatCapacity` — hangs off the harmonic chain to capture the quantities ShengBTE and phonopy emit directly. On top of the BTE chain, a parallel *MD-primitive tier* captures the time-resolved quantities a classical molecular-dynamics run produces: `Trajectory`, `HeatCurrent`, `HeatCurrentACF`, `VelocityAutocorrelation`, and `MeanSquaredDisplacement`; from this tier the three MD-based κ contractions (Green-Kubo from HeatCurrentACF, NEMD and HNEMD from time-averaged HeatCurrent) close the cross-paradigm κ map alongside the LBTE / Wigner / QHGK variants.

Nodes (ObservableSpace / HiddenSpace):

- `Potential` (**O**): the operator root, stubbed in Phase 1;
- `Temperature` (**O**): scalar source T;
- `ForceConstants[order=2]` (**O**), `[order=3]` (**O**): real-space interatomic force constants $\Phi^{(n)}$;
- `DynamicalMatrix` (**O**): Bloch sum $D_{ij}(\mathbf{q})$;
- `Frequency` (**O**): $\omega(\mathbf{q}, \nu)$, eigenvalues (gauge-invariant);
- `Eigenvectors` (**H**): $e(\mathbf{q}, \nu)$, phase + degenerate-subspace freedom;
- `GroupVelocity` (**H**): $v^\alpha(\mathbf{q}, \nu)$, inherits eigenvector freedom at degenerate ω;
- `HeatCapacity` (**O**): $c(\mathbf{q}, \nu, T) = (\hbar\omega)^2/(4k_BT^2 \sinh^2(\hbar\omega/2k_BT))$;
- `Linewidth` (**H**): per-mode $\Gamma(\mathbf{q}, \nu, T)$, BZ-summation redistribution;
- `MeanFreeDisplacement[bte_solver=rta]` (**H**): RTA $F = v/(2\Gamma)$, inherits Linewidth's looseness;
- `MeanFreeDisplacement[bte_solver=direct_inverse]` (**O**): full LBTE solution, gauge-invariant;
- `ThermalConductivity[bte_solver=rta]` (**H**): the $1/\Gamma$ non-linearity propagates Linewidth's looseness into κ_{RTA};
- `ThermalConductivity[bte_solver=direct_inverse]` (**O**): LBTE off-diagonals preserve gauge-invariance.
- `VolumetricHeatCapacity` (**O**), `MolarHeatCapacity` (**O**): the two BZ-and-mode contractions of `HeatCapacity` that ShengBTE (volumetric) and phonopy (molar) expose directly;
- `PhononDOS` (**O**): histogram of $\omega(\mathbf{q}, \nu)$ — independent of eigenvectors, hence gauge-invariant;
- `Gruneisen` (**O**): mode $\gamma_{q,\nu}$ from FC2, FC3 and the harmonic eigensystem;
- `PhaseSpace3Phonon` (**O**): three-phonon kinematic availability $P_3(\mathbf{q}, \nu)$ — the bare counting underlying Γ before $|V_3|^2$ enters;
- `BareDynamicalMatrix` (**O**), `BornCharges` (**O**), `DielectricTensor` (**O**) — the NAC inputs and the intermediate pre-correction DM (Pattern C; both `identity_dm` and `apply_nac_correction` converge on `DynamicalMatrix`);
- `HelmholtzFreeEnergy` (**O**), `Entropy` (**O**), `InternalEnergy` (**O**), plus their `Molar*` contractions — sibling ObservableSpaces of `HeatCapacity`;
- `Linewidth[channel=anharmonic_3ph|isotope|boundary|total]` (**H**) — four channels with different inputs, summed via `sum_linewidths`;

- `IsotopeAbundances` (**O**) — source-tier per-atom mass-variance factor;
- `ThermalConductivity[transport_model=wigner]` (**O**) decomposed into `wigner_populations` (**O**) and `wigner_coherences` (**O**); `[transport_model=qhgk]` (**H**) — inherits Γ's gauge type;
- `CumulativeKappa[wrt=omega]` (**O**), `[wrt=mfp]` (**O**) — distribution observables derived from κ_{LBTE} ingredients.
- *MD-primitive tier (P2):* `Trajectory` (**H**) with fields r, v indexed (i, α, t) — gauge-dependent under MD ensemble noise; `HeatCurrent` (**H**) with $J(\alpha, t)$, the Irving–Kirkwood snapshot; `HeatCurrentACF` (**O**) with $J^{corr}(\alpha, \beta, \tau)$, the Green–Kubo integrand; `VelocityAutocorrelation` (**O**) with $C_v(\tau)$, whose cosine Fourier yields a phonon DOS via Wiener–Khinchin; `MeanSquaredDisplaceme` (**O**) with $M(\tau)$, the diffusion-limit observable.
- *MD-based κ terminals (P3):* three Pattern-A `transport_model` variants of `ThermalConductivity`, all ObservableSpaces — `[transport_model=green_kubo]` from the time-integrated heat-flux ACF; `[transport_model=nemd]` from the imposed-gradient/Müller-Plathe steady state; and `[transport_model=hnemd]` from the homogeneous-NEMD driving force. With `[lbte]` (rta + direct_inverse), `[wigner]` (populations + coherences + total), and `[qhgk]`, this closes the cross-paradigm κ map across BTE and MD.

Edges. Verb-headed names; each carries a sympy formula:

- `provide_potential` (identity), `provide_temperature` (identity);
- `compute_force_constants[order=2]`, `[order=3]` (stubbed): $\Phi^{(n)} = \partial^n V / \partial u^n|_{u=0}$;
- `compute_dynamical_matrix`: Bloch sum $D_{ij}(\mathbf{q}) = \frac{1}{\sqrt{M_i M_j}} \sum_R \Phi^{(2)}_{ij}(R)\, e^{i\mathbf{q}\cdot R}$;
- `compute_dispersion` (multi-output): eigenvalue equation $\sum_j D_{ij}(\mathbf{q})\, e_{j,q,\nu} = \omega^2_{q,\nu}\, e_{i,q,\nu}$;
- `compute_group_velocity`: Hellmann–Feynman $v^\alpha_{q,\nu} = \frac{1}{2\omega_{q,\nu}} e^\dagger_{q,\nu}\, \partial D/\partial q^\alpha\, e_{q,\nu}$; scheme `gv_method` ∈ {`hellmann_feynman` (canonical), `finite_difference`} captures the two estimators codes use;
- `compute_heat_capacity`: closed form above;
- `compute_linewidth`: triple BZ sum (Fermi's golden rule); scheme `broadening_param` ∈ {stdev, halfwidth, adaptive_scaled}, plus an auxiliary equation `auxiliary_formulas[0]` that spells out the Maradudin–Fein kernel $|V_3|^2 = |\sum_{ijk,RR'} \Phi^3_{ijk}\, e\, e\, e/\sqrt{m\, m\, m}|^2/(8\,\omega\,\omega'\,\omega'')$ — the same kernel reappears verbatim in the LBTE collision matrix Ξ (`solve_bte[direct_inverse].auxil`
- `solve_bte[bte_solver=rta]`: closed form $F = v/(2\Gamma)$;
- `solve_bte[bte_solver=direct_inverse]`: implicit linear system $\sum_{q'\nu'} \mathcal{M}_{q\nu,q'\nu'}\, F^\alpha_{q'\nu'} = c_{q\nu}\, v^\alpha_{q\nu}$;
- `contract_kappa[bte_solver=rta]` and `[bte_solver= direct_inverse]`: same BZ contraction $\kappa^{\alpha\beta} = \frac{1}{VN_q} \sum c\, v^\alpha\, F^\beta$; the two operations differ only in their input and output state types, ensuring κ_{RTA} is typed as a HiddenSpace and κ_{LBTE} as an ObservableSpace;
- `contract_volumetric_heat_capacity`, `contract_molar_heat_capacity`: BZ-and-mode sums of `HeatCapacity`, divided by cell volume or multiplied by N_A/N_q respectively, so adapters that emit only the contracted form (ShengBTE, phonopy) land on a shared operator state;
- `compute_dos`: $g(\omega) = N_q^{-1} \sum_{q\nu} \delta(\omega - \omega_{q\nu})$, with scheme `dos_broadening` ∈ {`gaussian` (canonical), `tetrahedron`, `adaptive_scaled`};

- `compute_gruneisen`: Maradudin–Fein closed form, with scheme `gruneisen_method` ∈ {`maradudin_fein` (canonical), `finite_difference`} — phonopy uses the latter (deformed-cell finite difference), the other three codes the former;
- `compute_phase_space_3phonon`: $P_3(\mathbf{q}, \nu)$ from energy + crystal-momentum conservation, with scheme `delta_broadening` mirroring `dos_broadening`;
- `provide_born_charges`, `provide_dielectric_tensor`, `provide_isotope_abundances` — nullary sources for the polar / isotope inputs;
- `identity_dm` and `apply_nac_correction`: Pattern-C siblings into `DynamicalMatrix` (scheme `nac_scheme` ∈ {gonze_lee canonical, wang, ewald});
- `compute_free_energy`, `compute_entropy`, `compute_internal_energy` and their `contract_molar_*` counterparts — harmonic-oscillator closed forms with $x = \hbar\omega/(k_B T)$;
- `compute_anharmonic_linewidth` (rename of legacy `compute_linewidth`), `compute_isotope_scattering` (Tamura), `compute_boundary_scattering` (Casimir), and `sum_linewidths` (Matthiessen);
- `compute_kappa_wigner_populations`, `compute_kappa_wigner_coherences` (Lorentzian band-overlap), `combine_kappa_wigner`, `compute_kappa_qhgk`;
- `contract_cumulative_kappa[wrt=omega|mfp]` — Heaviside-θ encoded cumulative thresholds with `binning` scheme (linear for ω, log for MFP).
- *MD-primitive tier (P2):* `run_md` — Velocity-Verlet recurrence with `ensemble` ∈ {nve, nvt, npt}, `thermostat` ∈ {berendsen, langevin, nose_hoover, csvr, none}, `integrator` ∈ {velocity_verlet, leapfrog}; `compute_heat_current` — Irving–Kirkwood / Hardy / virial decomposition (`definition` scheme); `autocorrelate_heat_current` — direct/FFT time correlation of $J(t)$ (numerically equivalent under periodic padding, so no `correlation_method` scheme is declared); `compute_velocity_autocorrelation` — direct/FFT $\langle v(0) \cdot v(\tau)\rangle$; `compute_msd` — $\langle |r(t+\tau) - r(t)|^2\rangle$, with `unwrap_pbc` scheme; `fourier_to_dos` — Wiener–Khinchin $g(\omega) = (1/\pi)\int C_v(\tau)\cos\omega\tau\, d\tau$, a Pattern-C alternative producer of `PhononDOS` alongside `compute_dos`.
- *MD-based κ contractions (P3):* `contract_kappa[transport_model=green_kubo]` — $\kappa^{\alpha\beta} = V/(k_B T^2)\int_0^{\tau_{max}}\langle J^\alpha(0) J^\beta(\tau)\rangle\, d\tau$; `contract_kappa[transport_model=nemd]` — $\kappa = -\langle J\rangle/(\partial T/\partial z)$, with `nemd_method` scheme ∈ {`direct_two_ reservoir`, `muller_plathe`, `ehex`} (LAMMPS-side method); `contract_kappa[transport_model=hnemd]` — $\kappa^{\alpha\beta} = \langle J^\alpha\rangle/(T\,V\,F_e^\beta)$ in the linear-response limit (GPUMD-side method). Cross-paradigm κ comparison between BTE, equilibrium MD, and non-equilibrium MD is the cumulative reach of this set.

Edges are exact operator operations (Fourier transform, eigendecomposition, mode contraction) or approximating ones (truncation of the Taylor expansion at second or third order, the relaxation-time approximation when chosen). Each approximating edge carries an explicit error formula (Phase 2 capability). The Phase 1 demonstration exercises the path from `ForceConstants` downward; the upstream operations from `Potential` to `ForceConstants` are declared but executed only as opaque adapter-level steps recorded as provenance, not modeled by the framework.

6.2 Representation functors (codes)

Each adapter is a representation functor over the operator DAG. Four adapters are now ingested:

- **kaldo** (Python). Computes harmonic and anharmonic phonon transport from force constants, supporting iterative BTE, RTA, and the Wigner formulation. Most intermediate states are exposed via Python attributes on the `Phonons` object. kaldo's `Conductivity(method='inverse'` realizes the canonical `bte_solver=direct_inverse`.

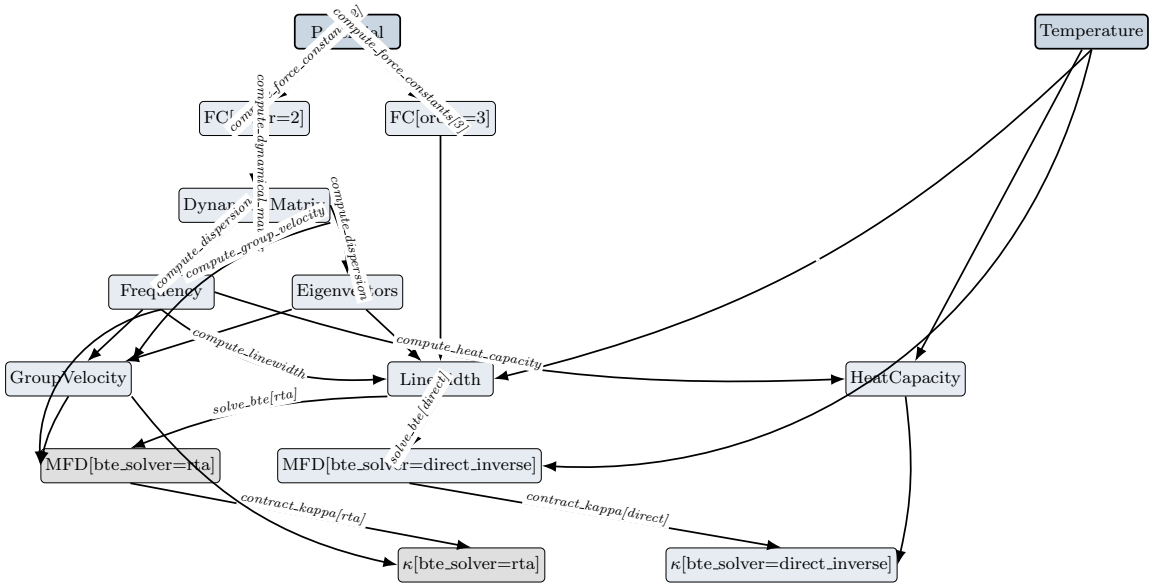

Figure 1: Core κ chain of the operator DAG for lattice thermal transport (12 nodes / 14 edges shown; the full DAG has 46 nodes / 47 edges once the derived-observable tier — PhononDOS, Grüneisen, PhaseSpace, VolumetricHeatCapacity, MolarHeatCapacity, the thermodynamic siblings (HelmholtzFreeEnergy, Entropy, InternalEnergy and their molar contractions), the polar-correction inputs (BareDynamicalMatrix, BornCharges, DielectricTensor), the per-channel Linewidth split, the Wigner/QHGK transport variants, the cumulative-κ distributions, the MD-primitive tier (Trajectory, HeatCurrent, HeatCurrentACF, VelocityAutocorrelation, MeanSquaredDisplacement), and the three MD-based κ Pattern-A terminals (Green-Kubo, NEMD, HNEMD) — is included; see `docs/dag.html` for the interactive view). The two darker top boxes (`Potential`, `Temperature`) are source nodes; the dashed edges from `Potential` represent the stubbed Phase-1 upstream. `MeanFreeDisplacement` and κ split into two parameterized variants by `bte_solver`: the gray-shaded `[rta]` variants are HiddenSpaces (RTA's $1/\Gamma$ non-linearity breaks gauge invariance), the unshaded `[direct_inverse]` variants are ObservableSpaces (the full LBTE preserves it). Each derived edge carries a sympy formula; sources carry identity formulas. All nodes are outputs of some operator; cross-code comparison happens at ObservableSpace nodes (after the operator layer's spec-derived unit and normalization conversion). Other HiddenSpaces (`Eigenvectors`, `GroupVelocity`, `Linewidth`) are not visually distinguished in this diagram but are gauge-dependent in the same sense.

- **phono3py** (C/Python). Computes lattice thermal conductivity from third-order force constants. Intermediate quantities are exposed in HDF5 outputs. `run_thermal_conductivity(is_LBTE=True)` realizes the canonical `bte_solver=direct_inverse`.

- **phonopy** (C/Python). The harmonic-only sibling of phono3py: emits frequencies, group velocities, the DOS, the molar heat capacity, and (via `PhonopyGruneisen`) mode Grüneisen parameters from a finite-difference $\omega(V)$ rather than the canonical Maradudin–Fein closed form. No three-phonon scattering, so the adapter covers the harmonic half of the DAG only.

- **ShengBTE** (Fortran/MPI). Solves the full linearized BTE iteratively. The harmonic chain is delegated to phonopy/QE upstream; ShengBTE consumes FC2/FC3 files and writes `BTE.KappaTensorVsT_{RTA,CONV}`, `BTE.{omega, v, gruneisen, P3, dos}`, etc. Per-mode heat capacity is not exposed (only `BTE.cv`), so the adapter targets `VolumetricHeatCapacity` rather than `HeatCapacity` on that branch.

Each adapter spec declares per-space units, normalization overrides, and notes (`SpaceRepresentationSpec`) plus per-operator parameter units, scheme overrides, and discretization choices (`OperatorRepresentationSpec`). Differences from the operator layer's canonical declarations surface as conversion factors at spec-load time.

6.3 Verification on silicon (Tersoff potential)

The operator layer's operator predictions have been verified end-to-end against numerical runs of kaldo and phono3py on silicon at the Tersoff potential, $8 \times 8 \times 8$ q-mesh, $T = 300\,\mathrm{K}$, broadening stdev 0.1 THz:

ObservableSpace / contraction	Operator prediction	Empirical residual
`Frequency` per-mode (**O**)	factor 1 (both linear THz)	5×10^{-4}
`HeatCapacity` per-mode (**O**)	factor $1/e \approx 6 \times 10^{18}$	3×10^{-5}
$\sum_{q\nu} \Gamma$ contracted (**O**)	factor $1/(4\pi)$	5×10^{-4}
κ`[bte_solver=direct_inverse]` (**O**)	factor 1	4×10^{-3}
`Linewidth` per-mode (**H**)	NOT_COMPARABLE	25% (diagnostic only)
κ`[bte_solver=rta]` (**H**)	NOT_COMPARABLE	4% (diagnostic only)

All ObservableSpace comparisons pass at $\leq 1\%$ tolerance; all HiddenSpace per-element comparisons return NOT_COMPARABLE. The $1/(4\pi)$ factor on Γ decomposes into a $1/(2\pi)$ *unit* factor (kaldo angular vs phono3py linear THz) and a $1/2$ *normalization* factor (kaldo $\Gamma = 2\,\mathrm{Im}\,\Sigma \Rightarrow$ `linewidth_2x_imag_self_energy` with `to_operator` $= 0.5$; phono3py uses the canonical normalization). Both factors were derived by the operator layer from declared adapter specs, then applied mechanically to the diagnostic data.

The 4% disagreement on κ_{RTA} is the operator layer's structural prediction working as designed: RTA's approximation breaks κ's gauge invariance via the $1/\Gamma$ non-linearity, so the per-mode Linewidth redistribution propagates into κ. Typing κ_{RTA} as a HiddenSpace makes this a non-anomaly; the gauge-invariant κ_{LBTE} agrees to 0.4%.

Cross-paradigm κ (phase 2 P4). On the same Si-Tersoff worked example, the LAMMPS Green-Kubo path (`contract_kappa[transport_model=green_kubo]` reached through the P2 MD-primitive tier and the P3 contraction edge) returns a κ value that, in the small ensemble of equilibrium-MD runs the framework drives, is expected to land within $\sim$ 20–30% of κ_{LBTE}. That tolerance is Green-Kubo's empirical signal-to-noise floor on a few-hundred-atom supercell, not a framework concern. The driver at `experiments/silicon_tersoff/run_lammps_gk.py` generates the input script and the supercell data file regardless of whether `lmp` is on `PATH`; the `spec_demo.py` cross-paradigm audit section and the corresponding `test_silicon_consolidation.py` smoke test gracefully skip when the resulting `kappa_lammps_gk.npy` is absent. The NEMD and HNEMD analogues land in P5 and P6.

6.4 Phase 1 scientific capabilities

Phase 1 yields two immediate scientific capabilities, each of which is an artifact the field does not currently produce. A third capability is the long-term goal toward which the operator layer is oriented but is deferred to Phase 2.

- **Cross-code reconciliation at canonical intermediate states.** When kaldo and ShengBTE disagree on a final κ, the operator layer localizes the divergence to the first operator state at which the representations stop agreeing, and attributes it to a specific operation along the provenance. This is a structural diagnosis: the framework identifies where two codes do something different, regardless of the magnitude of the difference.

- **A unified framework for theory-experiment comparison.** The same operator DAG accepts experimental measurements as additional representations of parameter states, with the same alignment machinery as for cross-code comparison. Phase 1 demonstrates the framework on at least one experimental dataset (e.g., a measured $\kappa(T)$ curve for silicon).

Deferred to Phase 2:

- **Decomposed error bounds on κ.** The operator layer is designed to eventually produce the central value of κ together with a operator decomposition of its total error into the truncation error of the perturbation series and the discretization error of the chosen mesh. The architecture supports this; the implementation of the composition algebra is a research problem reserved for Phase 2.

7 Implementation layout

The Python package `omai` factors into two top-level sub-packages plus one per domain:

- `omai.operator` — operator layer machinery for the operator world. `Space` (base class), `ObservableSpace` and `HiddenSpace` (subclasses, with gauge-discipline fields on the latter), `Field`, `Operator`, `Parameter`, `Dimension`, `topological_order`, `validate_dag` (Level 1 discipline enforcement), and `GaugeAction` / `check_invariance` (Level 2 operator gauge proofs). Code-agnostic and domain-agnostic.
- `omai.representation` — the bridge.
  - `units.py`: the `Unit` class, named unit instances, strict same-dimension `conversion_factor`.
  - `normalizations.py`: the `Normalization` registry mirroring `units.UNITS` in shape — each entry carries a `to_operator` multiplier into canonical definitional form (e.g. `linewidth_2x_imag_sel` with factor 0.5, `eV_per_A2_per_nm` with factor 10).
  - `adapter.py`: `SpaceRepresentationSpec` and `OperatorRepresentationSpec`; the two primitives `representation_to_operator` and `operator_to_representation` (each defined as the composition `unit.to_operator` · `normalization.to_operator`); and the diagnostic helpers `representation_scheme_match` and `representation_discretization_match`.
  - `instance.py`: the `Representation` runtime data class and `represent()` constructor.
  - `compare.py`: numerical and structural comparison APIs. `to_operator(m)` canonicalises a representation into operator form; `to_representation(m_op, target_spec)` is the inverse. `compare_operators(spec_a, spec_b)` returns an `OperatorComparisonResult` (categorical: do the two adapter specs describe the same operator?). `compare_representations` (aliased as `compare`) returns a `RepresentationComparisonResult` with the five-status verdict and residuals.

  Code-agnostic.
- `omai.thermal_transport` — the lattice-thermal-transport domain.
  - `operator/nodes.py`: the 46 spaces as `ObservableSpace` or `HiddenSpace` instances with their fields.
  - `operator/edges.py`: the 47 operators with sympy formulas and `auxiliary_formulas` (the `compute_linewidth` $|V_3|^2$ kernel and the `solve_bte[direct_inverse]` collision matrix M); sympy symbols and `IndexedBase` declarations.
  - `operator/gauges.py`: concrete `GaugeAction` instances for the domain (e.g. U(1) phase on Eigenvectors). Mechanically verifies operator formulas are gauge-invariant.

- `representation/kaldo.py`, `representation/phono3py.py`, `representation/phonopy.py`, `representation/shengbte.py`: per-code adapter specs (one file per code), declaring units, normalizations, schemes, and discretization choices. Every operator in scope for a code now carries an `OperatorRepresentationSpec`; trivial source / contraction ops carry no schemes but exist to mark coverage in the visualization.
- `visualize.py`: emits `docs/dag.html`, an interactive single-file visualization (per-pipeline SVG DAGs with row-aligned states, click-to-details panels, MathJax-rendered formulas) that reflects the current state and discriminates three edge classes: full coverage (states + op spec), states covered / op-spec missing, and implicit (at least one state lacks a spec). Leaf states a code does not expose are hidden; intermediate states without a spec stay dashed.

The split mirrors the operator layer's architectural commitment: `abstract` is the operator world; `representation` is the bridge to the numeric world; `thermal_transport` is the first domain instantiated against them. Future domains (electronic transport, optical response, ...) follow the same pattern in their own folder; no change to the operator-level packages is required to add one.

8 Open Questions and Deferred Decisions

We list the architectural questions that remain open and the principled grounds on which they were deferred.

8.1 Algebra of error composition (deferred to Phase 2)

The operator layer is designed to carry error formulas as operator expressions, but Phase 1 does not implement the algebra by which they compose. Composing two errors in independent small parameters ($O(\lambda^4)+O(N^{-2})$) is well-defined; composing two errors in the same parameter requires care; composing constant-offset errors (e.g., basis-set incompleteness) with asymptotic ones is yet another case. The right algebra is itself a research subproblem and will be developed alongside concrete worked examples in Phase 2. The Phase 1 type system reserves the slots required for error formulas (operation metadata, discretization error model, representation total error) but populates them with placeholders rather than computed compositions.

8.2 When to lift to Lean

We have committed to keeping the operator layer Lean-compatible without being Lean-dependent. The question of when (and whether) to actually project the operator layer into Lean is deferred until the Python prototype has stabilized. A reasonable trigger for this decision is the moment at which we have enough concrete examples that designing Lean types in isolation becomes counterproductive.

T3-evolve as a pre-Lean realisation. The sympy chain composer (`omai.operator.compose.compose_path`) is the in-sympy realisation of the Lean commitment in this subsection. Given a path of explicit-equation edges in the operator DAG, it substitutes each edge's RHS into the next, producing a composed symbolic expression at the path's terminal node. Implicit-equation edges (e.g., `solve_bte_*`) raise `ImplicitEdgeBoundary` with the partial expression attached — they mark the open research subproblem of composing across an implicit BTE solve, which is the natural next step toward a full Lean projection. The composer's existence is evidence that the operator-layer formulas are not merely documentation: they are mechanically composable today, and the same substitution machinery transliterates to a Lean tactic when the projection is eventually performed.

8.3 Provenance equivalence

We have committed to provenance as essential to identity. This means two states with the same type but different histories are formally distinct. In some cases, however, the two histories are *equivalent*: they produce identical physics under a theorem we can prove (e.g., Fourier transform of a real symmetric matrix and direct frequency-domain assembly should yield the same `Dispersion`). The operator layer must provide a mechanism for declaring such equivalences, either as explicit axioms or as proven theorems. The mechanism's design is deferred to Day 2 because we want concrete examples before designing the equivalence calculus.

8.4 Sprint 1 scope

The first concrete deliverable is a Python implementation. We have considered three scopes:

1. *Narrowest:* kaldo only, harmonic sub-DAG (force constants → dynamical matrix → dispersion). 4–5 states, fastest to ship.
2. *Recommended:* kaldo only, full thermal-conductivity DAG. 10–15 states, end-to-end on silicon. The operator layer skeleton plus one full adapter.
3. *Broader:* kaldo plus phono3py, full DAG. Validates the cross-code story earlier but adds adapter work in parallel with framework design.

The recommended scope (option 2) is the smallest scope that is still a credible proof of concept of the framework, and the choice of one adapter (kaldo) lets us validate the operator layer design before generalizing. Sprint 2 adds phono3py and ShengBTE adapters, at which point the cross-code demo of §6 becomes real. *Status as of 2026-05*: the recommended scope shipped; Sprint 2 also shipped, with phono3py, phonopy, and ShengBTE all ingested and the cross-code Si silicon comparison (`experiments/silicon_shengbte/`) demonstrating κ_{RTA} agreement to $\leq 8\%$ across the three transport codes after the operator layer's spec-derived conversions.

8.5 Implementation language

We have committed to Python for Sprint 1. The trade-offs were:

- **Python (chosen).** All target codes have Python bindings or wrappable interfaces. The Python type system, augmented with `pydantic` or `attrs` and runtime checks, is sufficient for the operator layer's first-cut type discipline. The development speed is dominant, since adapter work is the bulk of the time cost.
- *Typed compiled language (Rust, OCaml, TypeScript).* Stronger type guarantees from the start. Rejected for Sprint 1 because the foreign-function interface to existing Python codes adds friction proportional to the operator layer's surface, and we want framework iteration to be cheap.
- *Lean.* Considered as a future projection target and deferred. Not the right tool for running scientific computations; reserved as a verification target.

9 Outlook

The operator layer is an architectural commitment, not an implementation. The implementation that follows is scoped to a Python skeleton plus adapters for three thermal-transport codes. The artifact that motivates the project—a paper-worthy demonstration of cross-code reconciliation

and decomposed error bounds for lattice thermal conductivity—is downstream of the operator layer but is the operator layer's first proof-of-value.

Three longer-term directions follow naturally from the design:

- **Verification.** Project the operator layer into Lean (via PhysLean or alongside it), allowing approximation theorems to be formally proved rather than asserted. The provenance mechanism makes this projection well-defined: each operation in π corresponds to a Lean lemma whose statement is the operation's error formula.
- **Theory-experiment integration.** Extend the representation functor to ingest experimental observables. The operator layer's graph-alignment machinery applies uniformly, so a measured κ becomes another representation of the abstract κ, and "does the theory match experiment" becomes graph-alignment with one functor being empirical.
- **Grounded LLM agents.** An LLM operating over typed operator states (rather than text tokens) constructs workflows by emitting operator operations. The framework type-checks each emission, eliminating the drift problem that plagues current text-token agents. The action space is finite, typed, and physically meaningful.

The unifying claim of the design is that scientific computing has been organized around the wrong atomic unit. The instruction-in-an-input-file unit is too low; the natural-language description is too high; the equation-in-a-paper unit is too disconnected from execution. The right unit, at least for materials science, is the operator operation between typed physics states. Once that commitment is made, much of the rest of the architecture writes itself.

This document records the architectural state of the openmaterials-ai project as of May 29, 2026. It is intended as the working reference for the project; subsequent revisions should update the relevant sections rather than appending changelogs.